

Proiect VLSI

Boncan Dragoș & Ionescu Petru

May 12, 2026

1 Poarta NAND

Este formată din doi tranzistori PMOS în paralel și doi tranzistori NMOS în serie.

Dacă unul din inputuri este 0, atunci PMOS-ul corespunzător va conduce 1 către out, pe când NMOS-ul va fi blocat. Valoarea celuilalt input nu contează, dat fiind că amândouă NMOS-urile trebuie deblocate pentru a transmite 0, dar un singur PMOS e suficient pentru a transmite 1.

Rămâne cazul în care ambele inputuri sunt 1: PMOS-urile sunt blocate, pe când ambele NMOS-uri transmit de la masă, deci pe output avem 0.

2 Poarta NOT

2.1 Folosind poarta NAND

Legarea inputurilor ne lasă cu relațiile $\text{NAND}(0, 0) = 1$ și $\text{NAND}(1, 1) = 0$, echivalând o poartă NOT.

2.2 Cu un PMOS și un NMOS în serie

Pentru input 0, NMOS este blocat, dar PMOS transmite 1. În schimb, pentru input 1, NMOS leagă outputul la masă iar PMOS este blocat.

Astfel, este îndeplinită funcția de poartă NOT.

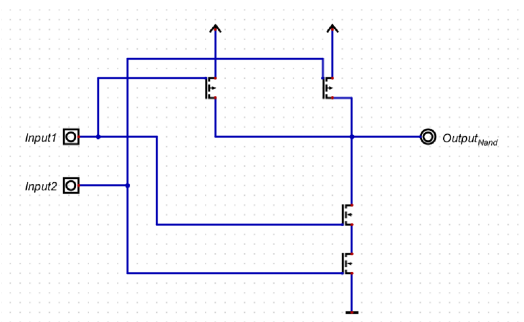


Figure 1: Poarta NAND

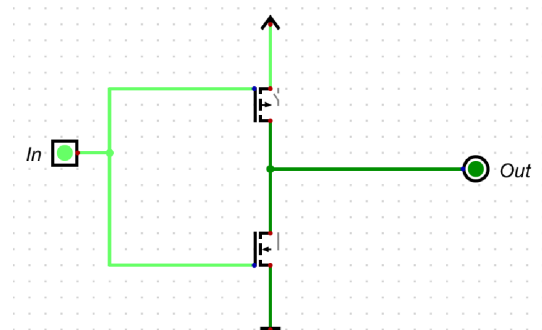


Figure 2: Poarta NOT

3 Registru pe 1 bit

A fost implementat cu master-slave, diferit de schema oferită, dar cu aceleași principii. În desenul original, se pot observa două căi.

Dacă S este 1, atunci tranzistoarele comandate de S sunt deschise, deci calea de sus e activă, scriind inputul în output. Altfel, calea de jos e cea activă, iar outputul se alimentează singur.

Se pot observa în circuitul nostru, de asemenea, EN și CLK, având următoarea relație:

$$S = \text{AND}(\text{EN}, \text{CLK}).$$

4 Registru pe 8 biți

Schema pentru registrul pe 8 biți este simplă odată având construit registrul pe 1 bit, trebuind numai să reproducem circuitul de 8 ori.

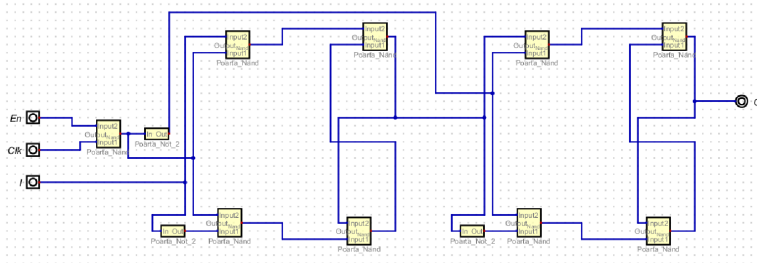


Figure 3: Registru pe 1 bit

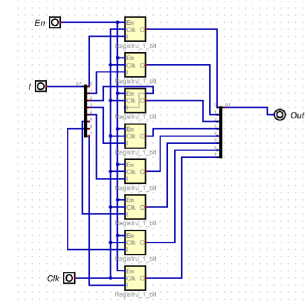


Figure 4: Registru pe 8 biți

5 Decodificator 2x4

În esență, decodificatorul primește o adresă și o mapează pe registrul dorit prin operațiile următoare:
 $\text{AND}(\text{NOT}(i0), \text{NOT}(i1))$ către out0
 $\text{AND}(\text{NOT}(i0), i1)$ către out1
 $\text{AND}(i0, \text{NOT}(i1))$ către out2
 $\text{AND}(i0, i1)$ către out3
 S-au folosit, astfel, porțile NOT și NAND implementate mai devreme.

6 Multiplexor

Implementarea pe care am ales-o folosește transmission gates. Pentru $\text{SEL} = 0$, A este conectat cu output iar B nu, pe când pentru $\text{SEL} = 1$ doar B este conectat cu outputul.

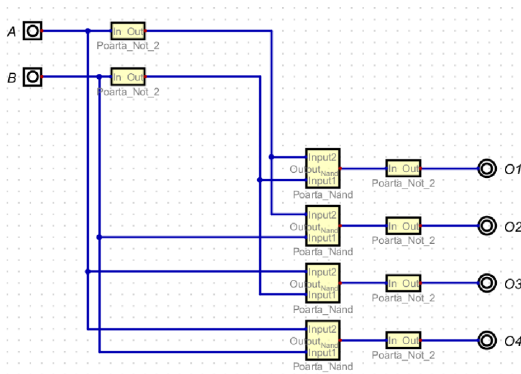


Figure 5: Decodificator

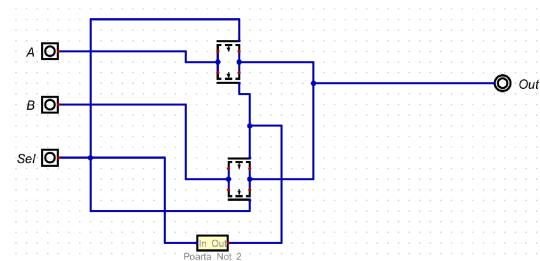


Figure 6: MUX 2x1 pe 1 bit

A fost făcută implementarea și pentru 8 biți, unde avem 8 MUXuri, fiecare cu biții corespunzători din A și B și cu același SEL.

În cadrul proiectului e nevoie de un MUX 8x1 pe 8 biți. Pentru acesta am folosit 7 MUXuri 2x1 pe 8 biți. În primele 4 MUXuri vin cele 8 intrări și primul (MSB) bit din SEL. Cele 4 outputuri pe care le obținem trec mai departe în două MUXuri setate de bitul din mijloc al lui SEL. În final, cele două outputuri obținute trec printr-un ultim MUX controlat de LSB din SEL și ne oferă rezultatul. De notat că SEL trebuie să fie pe 3 biți aici.

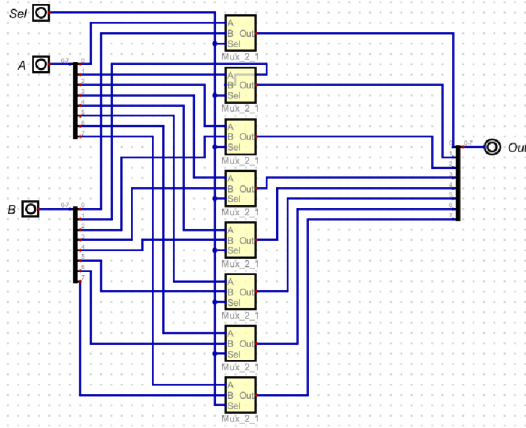


Figure 7: MUX 2x1 pe 8 biți

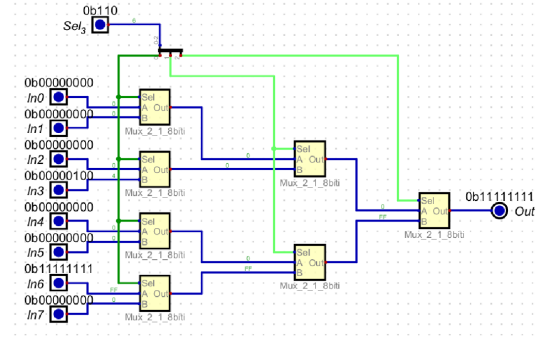


Figure 8: MUX 8x1 pe 8 biți

7 Poarta XOR

Am ales o implementare care folosește, pe lângă inversoare, patru transmission gates. Funcționează în felul următor: dacă $A = 0$, atunci se activează ramura controlată de $\text{NOT}(A)$ și se trimite B pe output. Altfel se activează cealaltă ramură și ieșirea primește $\text{NOT}(B)$.

Ecuția e echivalentă cu $\text{OR}(\text{AND}(A, \text{NOT}(B)), \text{AND}(\text{NOT}(A), B))$.

8 Sumator

Am folosit aceeași structură din document. În esență, urmărim ecuațiile:

$$S = \text{XOR}(\text{XOR}(A, B), \text{Cin}) \text{ și } \text{Cout} = \text{XOR}(\text{AND}(A, B), \text{AND}(\text{XOR}(A, B), \text{Cin}))$$

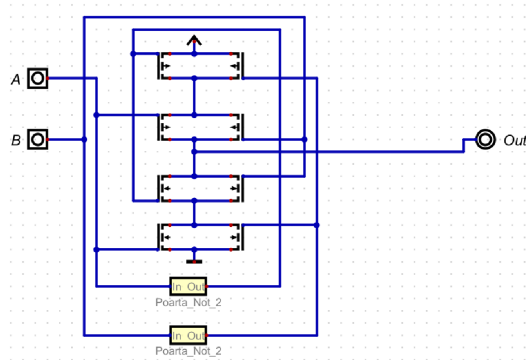


Figure 9: Poarta XOR

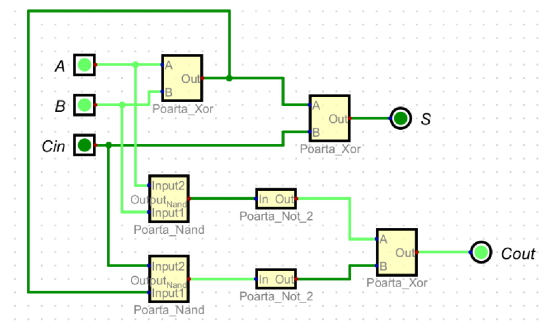


Figure 10: Sumator pe 1 biț

Mai departe, avem nevoie de această operație și pe 8 biți.

Soluția este să legăm opt sumatoare. Legarea se face în felul următor: Cout de la rezultatul sumei pe LSB se transmite în Cin pentru al doilea bit de la dreapta și tot așa, până Cout de la MSB este Coutul circuitului.

În figura de mai jos avem și un exemplu pentru a ilustra mai bine funcționalitatea.

9 Noter

Folosind circuitul sumator și pentru scădere, ne trebuie complementul față de 2 a unui număr, adică numărul negat la care e adunat 1.

Noterul are nevoie de un bit care specifică dacă vrem numărul negat (scădere) sau normal (adunare).

În circuit am denumit acest indicator drept Sub, Care face NOT(A) pentru Sub = 1 și returnează A pentru Sub = 0. Sunt 8 porți XOR, pentru fiecare bit din A, și fiecare primind ca unul din parametri valoarea lui Sub.

Noterul va fi folosit, evident, pe al doilea element al operației, acesta fiind scăzătorul în cazul Sub = 1.

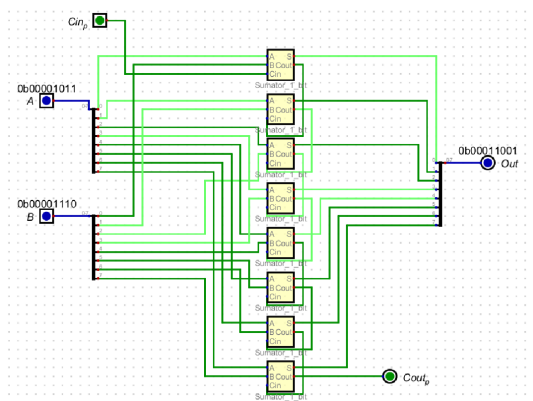


Figure 11: Sumator pe 8 biți

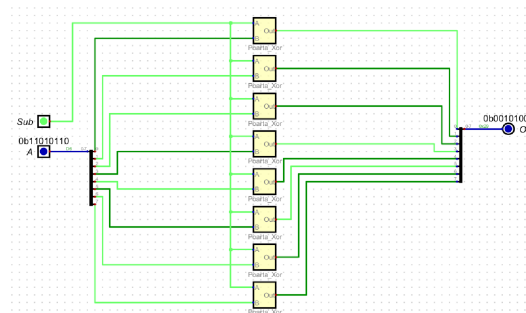


Figure 12: Noter

10 ALU

După crearea sumatorului și a noterului, putem asambla ALU. În documentație, ALU este văzut ca echivalent cu sumatorul, însă noi l-am făcut să cuprindă inclusiv noterul.

11 I/O

Nu am adăugat un keyboard, dar am creat un display. Outputul din ROM dictează ce va fi afișat. Mai în detaliu, fiecare bit din cei 14 spune dacă un bec este aprins sau nu.

Becurile sunt așezate în asemenea fel încât pot reprezenta orice număr de una sau două cifre. De exemplu, pentru afișarea lui 10, vom avea următoarele becuri aprinse: B2, C2 (acestea pentru cifra zecilor), A1, B1, C1, D1, E1 și F1 (acestea pentru cifra unităților).

Configurațiile care mapează un număr returnat la starea corespunzătoare a LEDurilor se află într-un ROM.

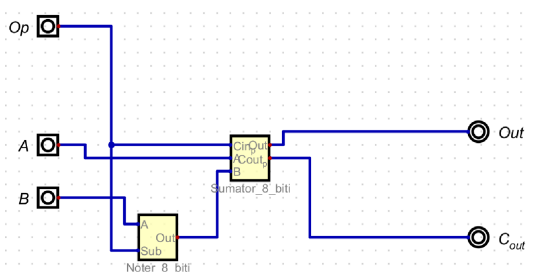


Figure 13: ALU

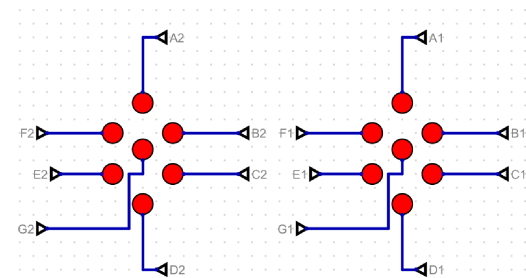


Figure 14: Display

12 Program counter

Program counter-ul este pe 8 biți și folosește un ALU pentru a realiza adunarea cu 1 (se poate vedea termenul B al adunării, cât și Op = 1, specificând adunare). Desigur, se putea folosi și componenta Sumator direct.

LDc este principalul indicator al operației făcute de PC, anume:

- ori incrementarea, care se întâmplă când LDc = 0, Count = 1, Clk = 1 și astfel MUXul alege valoarea A, adică valoarea din registrul de 8 biți trecută prin ALU
- ori copierea lui Din, atunci când LDc = 1

13 Unitatea de comandă

Unitatea de comandă are o multitudine de semnale, însumând un total de 20 de biți. Semnificația fiecăruia:

- Bitul 0 - X - nu e folosit
- Bitul 1 - CLC - semnalizează finalizarea instrucțiunii curente
- Biții 2-4 - OPC - indică operația executată de ALU; cum avem doar adunare și scădere, e nevoie doar de un bit (de MSB): 100 reprezintă scădere, pe când 000 adunare
- Bitul 5 - Flags - pe baza ultimei operații din ALU comandă actualizarea Carry
- Bitul 6 - IR.en
- Bitul 7 - OUT.en
- Bitul 8 - IN.en
- Bitul 9 - C.en
- Bitul 10 - B.en
- Bitul 11 - A.en
- Bitul 12 - LD_count - încarcă o valoare în Program counter (folosit la jump-uri)
- Bitul 13 - Count - incrementarea PC
- Bitul 14 - MAR.en
- Biții 15-17 - BUS - codifică sursa care poate plasa date pe magistrală
- Bitul 18 - LD - semnal de citire din RAM
- Bitul 19 - STR - semnal de scriere în RAM

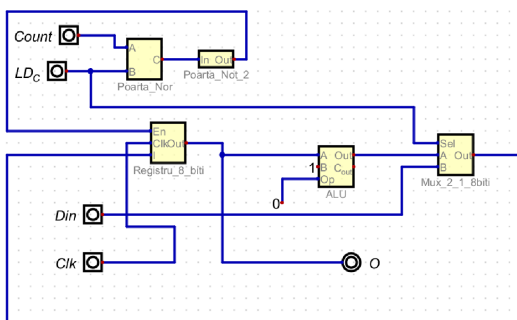


Figure 15: Program counter

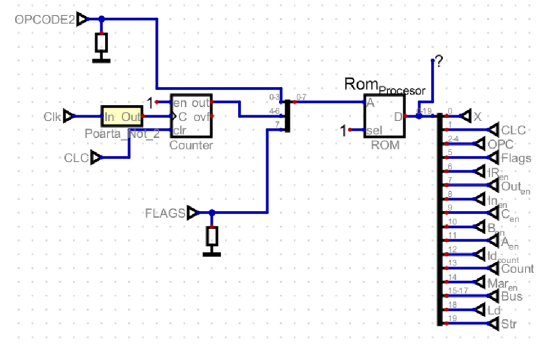


Figure 16: Unitatea de comandă

14 Instrucțiuni implementate

Au fost cerute următoarele instrucțiuni:

- 0000 - NOP
- 0001 - LDA [addr] - spre exemplu, 1a face load la adresa a din ram
- 0010 - LDA value - de exemplu, 23 face load la valoarea 3
- 0011 - MOV B, A - codificarea este 30
- 0100 - MOV C, A - se face cu 40
- 0101 - MOV A, B - codificat ca 50
- 0110 - MOV A, C - codificat ca 60
- 0111 - MOV OUT, A - codificat ca 70
- 1000 - STR A - store din adresa A la o adresă din RAM
- 1001 - STR valoare - store unei valori
- 1010 - ADD
- 1011 - SUB
- 1100 - JMP addr - pentru salturi necondiționate
- 1101 - JC addr - salturi condiționate

În plus, neutilizând un keyboard, am făcut interacțiunea mai lesne prin implementarea a două alte instrucțiuni și astfel se pot primi valori manual:

- 1110 - MOV A, In
- 1111 - MOV B, In

15 Întrebările din cerință

15.1 R1

În cazul în care ambele inputuri sunt 1: PMOS-urile sunt blocate, pe când ambele NMOS-uri transmit de la masă, deci pe output avem 0.

Dacă unul din inputuri este 0, atunci PMOS-ul corespunzător va conduce 1 către out, pe când NMOS-ul va fi blocat. Valoarea celuilalt input nu contează, dat fiind că amândouă NMOS-urile trebuie deblocate pentru a transmite 0, dar un singur PMOS e suficient pentru a transmite 1.

15.2 R2

Pentru varianta cu un NMOS și un PMOS în serie: dacă $a = 0$, atunci PMOS-ul transmite 1 ieșirii, pe când NMOS e blocat. Altfel, NMOS transmite 0, iar PMOS-ul e blocat.

În cel de-al doilea caz, legarea inputurilor ne lasă cu relațiile $\text{NAND}(0, 0) = 1$ și $\text{NAND}(1, 1) = 0$, echivalând o poartă NOT.

15.3 R3

Rolul lui S este de a marca dacă se scrie (1) sau se memorează (0).

Vedem că sunt două "căi", una în partea de sus a circuitului, iar cealaltă jos. Dacă $S=1$, atunci tranzistoarele comandate de S sunt deschise, iar cele comandate de NOT(S) închise, astfel având calea de sus activă (cea de scriere a lui i).

Analog vedem situația și pentru $S=0$, când calea de jos e activă. În cazul acesta outputul se alimentează singur și obținem rezultatul așteptat.

15.4 R4

Pentru $S=1$ vom avea $out=00101000$, iar pentru $S=0$ vom avea out -ul egal cu ultima valoare a sa. Dacă aveam $out=00101000$ înainte, atunci obținem tot 00101000 .

15.5 R5

S este $AND(EN, CLK)$, de unde putem trage concluzia că $S=1$ doar când și EN , și CLK sunt 1. Astfel, ieșirea e actualizată doar pe ciclul de ceas activ și prin activarea din EN .

15.6 R6

Conform tranzistoarelor, avem următoarele rezultate:

```
out0 = AND(NOT(i0), NOT(i1))
out1 = AND(NOT(i0), i1)
out2 = AND(i0, NOT(i1))
out3 = AND(i0, i1)
```

Evident, numai una din ieșiri poate fi 1 la un moment dat, deci decodificatorul își îndeplinește funcția de a activa o singură ieșire.

15.7 R7

În cadrul operației de scriere se întâmplă următoarele:

- încărcăm în MAR adresa unde vrem să scriem
- biții din MAR ajung la decodificatoare
- decodificatoarele activează o linie de memorie
- valoarea de pe $DATA_IN$ e aplicată pe magistrala de date
- dacă $EN=1$ și $CLK=1$, registrul selectat preia valoarea

15.8 R8

În cadrul operației de citire:

- punem în MAR adresa de unde vrem să citim
- decodificatorul activează linia corespunzătoare de memorie
- celula de memorie selectată pune valoarea pe intrarea MUX -ului
- MUX -ul selectează valoarea corespunzătoare
- valoarea ajunge pe $DATA_OUT$

15.9 R9

Intrarea de selecție (Sel) comandă care din celelalte două intrări (a sau b) va fi propagată către ieșire. Pentru $Sel = 0$, a este cel propagat, pe când, pentru $Sel = 1$, b este cel propagat.

Putem observa cum una din căi e controlată de $NOT(Sel)$, pe când cealaltă de Sel , asigurând că o singură cale e activă la un moment dat.

15.10 R10

Pentru calculul sumei se va trece prin următorii pași:

- adresa 8 e încărcată în MAR
- se citește valoarea 3 de la adresa 8
- 3 se încarcă în A
- analog, se încarcă adresa 9 în MAR, se citește valoarea 4, și 4 se încarcă în B
- ALU execută $A + B$
- rezultatul (7) se încarcă în registrul C

15.11 R11

Pentru LDA value, obținem opcode-ul 38800. Adresele corespunzătoare sunt 00100010 și 10100010 (având Cout = 0/1, ciclul de ceas 010 și 0010 codul operației). Astfel, vom avea scrisă valoarea 38800 în celulele 22 și A2

STR	LD	BUS	MAR	COUNT	LDC	A	B	C	IN	OUT	IR	Flags	OPC	CLC	X
0	0	111	0	0	0	1	0	0	0	0	0	0	000	0	0

Table 1: LDA value

Analog putem trece și la următoarele operații.

Opcode-ul corespunzător lui MOV B, A este 10400 și îl scriem în celulele 23 și A3 (echivalente cu 00100011 și 10100011 în binar).

STR	LD	BUS	MAR	COUNT	LDC	A	B	C	IN	OUT	IR	Flags	OPC	CLC	X
0	0	010	0	0	0	0	1	0	0	0	0	0	000	0	0

Table 2: MOV B, A

Pentru MOV C, A, opcode-ul este 10200 și este scris la adresele 24 și A4.

STR	LD	BUS	MAR	COUNT	LDC	A	B	C	IN	OUT	IR	Flags	OPC	CLC	X
0	0	010	0	0	0	0	0	1	0	0	0	0	000	0	0

Table 3: MOV C, A

Pentru MOV A, B, opcode este 18800 și, intuitiv, e scris la 25 și A5.

MOV A, C este codificat ca 28000 la adresele 26 și A6.

Pentru MOV OUT, A, avem 010 pe BUS și semnalul din OUT setat, deci opcode-ul este 10080, poziționat la adresele 27 și A7.

Mai departe, STR A este mai complexă, fiind formată din MAR j- IR și memory[MAR] j- A.

Pentru MAR j- IR, adresele în binar sunt 00101000 și 10101000, adică 28 și A8. La aceste adrese vom avea 3C000.

Pe de altă parte, la memory[MAR] j- A adresele sunt 00111000 și 10111000, adică 38 și A8. Observăm că avem ciclul de ceas următor, 011. La adresele 38 și A8 avem 90000.

STR valoare are opcode B8000 la adresele 39 și B9. Totodată, va pune și 3C000 (la fel ca STR A) la 29 și A9.

ADD și SUB se fac analog. ADD va scrie la 2A și AA pe 30200, iar SUB pe 30224 la 2B și AB.

Instrucțiunile cu salt vor seta LDCount.

Astfel, JMP are opcode 39000, pus la adresele 2C și AC.

JC se întâmplă doar când Cout = 1, deci opcode-ul 39000 va fi scris doar la adresa AD.

Mai departe, mai avem două instrucțiuni implementate în plus.

Prima din ele este MOV A, In, care e compusă din activarea In-ului (anume 0100 la adresele 2E și AE) și din încărcarea în A, având opcode 28800 la adresele 3E și BE (următorul ciclu de ceas).

STR	LD	BUS	MAR	COUNT	LDC	A	B	C	IN	OUT	IR	Flags	OPC	CLC	X
0	0	011	0	0	0	1	0	0	0	0	0	0	000	0	0

Table 4: MOV A, B

STR	LD	BUS	MAR	COUNT	LDC	A	B	C	IN	OUT	IR	Flags	OPC	CLC	X
0	0	100	0	0	0	1	0	0	0	0	0	0	000	0	0

Table 5: MOV A, C

MOV B, In va scrie și el 0100, dar la adresele 2F și AF. Totodată, asemănător cu instrucțiunea pe A, va scrie 28400 la 3F și BF.

15.12 R12

Programul ales este generarea șirului Fibonacci, având codul în Assembly și codificarea în memorie la finalul documentului.

STR	LD	BUS	MAR	COUNT	LDC	A	B	C	IN	OUT	IR	Flags	OPC	CLC	X
0	0	010	0	0	0	0	0	0	0	1	0	0	000	0	0

Table 6: MOV OUT, A

STR	LD	BUS	MAR	COUNT	LDC	A	B	C	IN	OUT	IR	Flags	OPC	CLC	X
0	0	111	1	0	0	0	0	0	0	0	0	0	000	0	0

Table 7: MAR j- IR

STR	LD	BUS	MAR	COUNT	LDC	A	B	C	IN	OUT	IR	Flags	OPC	CLC	X
1	0	010	0	0	0	0	0	0	0	0	0	0	000	0	0

Table 8: memory[MAR] j- A

STR	LD	BUS	MAR	COUNT	LDC	A	B	C	IN	OUT	IR	Flags	OPC	CLC	X
1	0	111	0	0	0	0	0	0	0	0	0	0	000	0	0

Table 9: STR valoare

STR	LD	BUS	MAR	COUNT	LDC	A	B	C	IN	OUT	IR	Flags	OPC	CLC	X
0	0	110	0	0	0	0	0	1	0	0	0	0	000	0	0

Table 10: ADD

STR	LD	BUS	MAR	COUNT	LDC	A	B	C	IN	OUT	IR	Flags	OPC	CLC	X
0	0	110	0	0	0	0	0	1	0	0	0	1	001	0	0

Table 11: SUB

STR	LD	BUS	MAR	COUNT	LDC	A	B	C	IN	OUT	IR	Flags	OPC	CLC	X
0	0	111	0	0	1	0	0	0	0	0	0	0	000	0	0

Table 12: JMP

STR	LD	BUS	MAR	COUNT	LDC	A	B	C	IN	OUT	IR	Flags	OPC	CLC	X
0	0	111	0	0	1	0	0	0	0	0	0	0	000	0	0

Table 13: JC

STR	LD	BUS	MAR	COUNT	LDC	A	B	C	IN	OUT	IR	Flags	OPC	CLC	X
0	0	000	0	0	0	0	0	0	1	0	0	0	000	0	0

Table 14: Activare In

STR	LD	BUS	MAR	COUNT	LDC	A	B	C	IN	OUT	IR	Flags	OPC	CLC	X
0	0	101	0	0	0	1	0	0	0	0	0	0	000	0	0

Table 15: MOV A, In

STR	LD	BUS	MAR	COUNT	LDC	A	B	C	IN	OUT	IR	Flags	OPC	CLC	X
0	0	101	0	0	0	0	1	0	0	0	0	0	000	0	0

Table 16: MOV B, In

Assembly	Codificare în memorie
LDA [b]	1b
MOV B, A	30
LDA [a]	1a
LOOP:	
ADD	a0
JC END	d9
MOV B, A	30
MOV A, C	60
MOV OUT, A	70
JMP LOOP	c3
END:	
NOP	00
a: DB 1	01
b: DB 1	01

Table 17: Șirul lui Fibonacci